

A arquitectura do software é a **descomposición** do software nun conxunto de **compoñentes/módulos** e **iteracción** entre eles. A definición da arquitectura dun software consiste en realizar a **selección** deses elementos que o integran e decidir a súa **organización**. A arquitectura do software debe permitir satisfacer os **requisitos non funcionais** do software, e sirva de base para o seu posterior deseño e implementación.

- O deseño software
 - Deriva dos **requisitos funcionais**
 - Organizámolo usando **patróns de deseño**
 - Pode despregarse en diferentes arquitecturas
- A arquitectura do software
 - Deriva dos **requisitos non funcionais**
 - Organizámolo usando **estilos** de arquitectura.

Definir a arquitectura é tomar as **decisións de máis impacto** que dan forma ao software, onde o impacto se mide en **custe de cambio**.

- Estructura, tecnoloxías, requisitos non funcionais.

Os **estilos de arquitectura** determinan o uso de certos elementos e o seu xeito de organizarse.

- Serven como mecanismo de comunicación entre desenvolvedores, ao igual que os patróns de deseño software.

A arquitectura do software represéntase habitualmente de xeito informal e heteroxéneo. Existe un estándar, o IEEE 1471, pouco adotado. Calquera representación debe:

- Indicar que é cada compoñente e o papel que xoga.
- Mostrar as interaccións entre compoñentes e o seu fin.
- Manter coherencia entre compoñentes semellantes.
- Manter coherencia entre interaccións equivalentes.

Modelo C4: a arquitectura do software represéntase mediante unha sucesión de catro vistas.

- **Contexto**, que mostra a relación do sistema con outros
- **Contedor**, que mostra a distribución física dos compoñentes.
- **Compoñente**, que mostra as relacións entre compoñentes.

- **Código** (*Diagramas de componentes UML*).

Definen **propiedades e restricións do sistema** (fiabilidade, tempo de resposta, capacidade de almacenamento, etc.)

- Poden incluír restricións de entorno e/ou negocio (tecnoloxías, ciclo de vida...).
- Afectan a todo o sistema e non só a compoñentes concretos.

Son **tanto ou máis críticos que os requisitos funcionais**, xa que de non cumprirse, o software pode resultar inútil.

- Poden xerar/restrinxir requisitos funcionais.



Adoitan ser máis complexos de **concretar**, pero se son imprecisos son difíciles de validar.

- Debemos asegurarnos de que os requisitos non funcionais son **verificables**

Deben validarse en conxunto para asegurar que son **consistentes e realistas**.

- Hai requisitos conflictivos entre si?
- Poden implementarse co tempo e os recursos dispoñibles?

Requisito non funcional	Medida obxectiva
Velocidade, rendemento	Transaccións procesadas por segundo Tempo de resposta a un evento Tempo de recarga da pantalla Tempo de reinicio despois dun fallo
Consumo de recursos	Memoria, capacidade de cómputo, enerxía
Facilidade de uso	Tempo de adestramento/resolución de tarefa(s), "memorabilidade", Número de páxinas de axuda

Requisito non funcional	Medida obxectiva
Fiabilidade/dispoñibilidade Robustez/tolerancia a erros	Tempo medio de fallo Ratio de ocorrencia de fallos Porcentaxe de eventos que provocan un fallo Probabilidade de corrupción de datos tras un fallo
Portabilidade	Número de plataformas obxectivo Porcentaxe de elementos dependentes da plataforma
Escalabilidade	Peticións procesadas por segundo sen variación significativa da velocidade/rendemento

Resilencia no software

- Disponibilidade
- Robustez
- Tolerancia a erros/fallos
- Fiabilidade
- Adaptabilidade
- Escalabilidade
- Rendemento

Resilencia

O sistema de **proporcionar**, de xeito observable, un **servizo** consistente coa súa especificación.

Cando un sistema **non proporciona un servizo consistente coa súa especificación**, percíbese como un **fallo**.

- O fallo é **observable** polos usuarios do sistema, sexan humanos ou outros sistemas.

A descrición deste parámetro determina cales son as **consecuencias aceptables** de que se presente un **fallo no sistema**.

Pódese medir como a fracción de **tempo que o sistema presta servizo** consistente coa súa especificación

- Unha *dispoñibilidade* de 0.998 significa que o sistema está listo 998 de cada 1000 unidades de tempo
 - Un 0.9 significa que está listo o 90% do tempo, isto é, pode caerse até 144 minutos cada 24h
 - Un 0.99 significa que pode caerse 14 minutos cada 24h
 - Un 0.999 significa 84 segundos por cada 24h
 - Un 0.9999 significa un minuto á semana
- Debe ter en conta o tempo necesario para reparar e reiniciar o sistema en caso de fallo.
- Preguntas chave
 - ¿Como se **identifica** un fallo?
 - Omisión / *crash* / tempo resposta / ...
 - Con canta **frecuencia** se permite? Canto **tempo** pode levar **recuperar** o sistema?
 - *Trecuperación, Tdispoñibilidade, Tdegradado, Tpreparación*

- **Que pasa** cando ocorre?
 - Notificar / continuar en operación / desactivar compoñente / deter o sistema / ...

Podemos diferenciar unha caída no **rendemento** dunha caída do sistema?

- **Resposta** do sistema a **eventos** temporais
- Debe realizarse nun **intervalo aceptable** (especificación)
- Preguntas chave
 - Como se **identifica** a caída do rendemento?
 - Lantecia (t en procesar un evento), *deadline*, *throughput* (eventos procesados en t), *jitter* (variabilidade do throughput), taxa de fallo (eventos, datos).
 - Que **tipo** de peticións se agardan?
 - Eventos periódicos / estáticos / esporádicos
 - **Que ocorre** cando se produce?
 - Notificar / continuar en operación / activar compoñente / rexeitar peticións / ...

Tácticas de resiliencia

Estratexias complementarias á arquitectura orientadas a *enviar a percepción de fallos do sistema*

- Incluído respostas demasiado tarde (rendemento).
Aadoitan involucrar mecanismos de monitorización, redundancia e prevención (sexa manual ou automatizada).

Tácticas de monitorización

- **Ping/echo**: incluír un compoñente que comproba a dispoñibilidade de outros facendo pong e agardando unha resposta nun tempo determinado.
- **Heartbeat** (dead man timer): os compoñentes emiten sinais periodicamente, que son escoitadas por outros; se un sinal non chega, o compoñente correspondente dáse por morto (e poden tomarse medidas).
- **Monitor de procesos**: proceso supervisor que detecta o mal funcionamento doutros. detén, reinicia, substitúe... compoñentes segundo o seu comportamento - ou cada certo tempo (prevén quedarse sen memoria por *memory leaks* presentes pero non detectados, buzóns cheos...) -
 - Comparación de patróns de "tráfico" (historicos).
 - Filtrado de peticións (procedencia, tipoloxía...)
 - Monitorización, almacenamiento, notificación

Tácticas de redundancia

- **Votación:** incluír compoñentes equivalentes que realizan a mesma operación en paralelo e un de control que comproba as súas respostas; se algún difire moito anúlase ou reiníciase; co resto tómase unha decisión.
- **Redundancia activa:** incluír compoñentes redundantes que procesan en paralelo; só se usa un (p.ex. o máis rápido); se ese falla, recórrase á resposta de outro.
- **Resposta pasiva:** incluír un conxunto de compoñentes dos que un responde aos eventos e infoma ao resto do resultado; se falla, un compoñente de control substitúeo por outro.
- **Reposto/replicación:** incluír compoñente equivalente que configurar e inicializar en caso de fallo do orixinal; *checkpoints* ou *backups* periódicos do orixinal con *log* de operación.
 - Réplicas para reducir o tempo de agarda por un recurso.
 - Se o recurso replicado son datos, o sistema é responsable de manter consistencia e sincronización.

Tácticas de prevención

- **Xestión da ratio de eventos**
 - Reducir a frecuencia coa que se tratan os eventos, descartándoos se non temos control sobre a súa ratio de chegada.
- **Control da frecuencia de mostraxe**
 - Reducir a frecuencia coa que se monitoriza o contorno para reducir o número de eventos.
- **Limitación do tempo de execución**
 - Limitar o tempo que se agarda pola resposta a un evento para controlar o uso de recursos
- **Limitación do tamaño dos buffers**
 - Limitar a cantidade de eventos pendentes de ser procesados para controlar o uso de recursos.

Arquitectura cliente-servidor

Baséase na organización de **funcionalidades como servizos independentes**, proporcionados por compoñentes específicos.

- Tamén se denomina arquitectura orientada a servizos (*service oriented architecture* - SOA), baseada en (micro)servizos.

A maior **vantaxe** é a robustez, xa que fallos nun servizo non afectan ao funcionamento do resto de servizos ofrecidos polo sistema. O maior **inconveniente** é que para que os clientes non se acoplen cos servizos, precisamos un directorio intermedio que pode converterse en limitante do rendemento.

Arquitectura cliente-servidor distribuído

Baséase, ao igual que a arquitectura cliente/servidor clásica, na organización de **funcionalidades como servizos independentes**, proporcionados por compoñentes específicos, e **despregados en máquinas diferentes**.

A maior **vantaxe** fronte á cliente/servidor clásica é que incrementa a súa robustez, escalabilidade e rendemento. O maior **inconveniente** segue a ser a necesidade dun directorio intermedio.

Preséntase en dúas variantes:

- **Cliente lixeiro**, encargado únicamente da representación da interface, e onde os servidores implementan toda a lóxica dos servizos.
 - A maior **vantaxe** é que son clientes fáciles de manter (importante se o número é elevado).
 - O maior **inconveniente** é que se intercambia gran cantidade de información pola rede.
- **Cliente pesado**, encargado de parte da lóxica dalgúns servizos, ademais da representación da interface.
 - A maior **vantaxe** é que aproveita a capacidade de proceso dos clientes.
 - O maior **inconveniente** é que o despregue e o mantemento requiren maior esforzo.

Caracterización da arquitectura cliente-servidor

Descrición: As funcionalidades do sistema se organizan en servizos, cada un deles implementado por un compoñente dedicado. Os clientes do sistema usan estes servizos de xeito transparente.

Aplicabilidade: Úsase cando se constrúen sistemas de xeito incremental, xa que non todos os

servizos teñen que estar rematados para poder ofertar os que si o están aos clientes. Tamén cando a heteroxeneidade dos servizos a ofertar non encaixa con outras arquitecturas.

Vantaxes: Os servizos son independentes entre si, polo que poden duplicar para facer fronte a alta demanda, e o fallo dun non debe afectar ao resto, de xeito que este estilo ofrece boa robustez. Tamén resulta doado ofrecer certos servizos a certos clientes, limitando a súa exposición.

Desvantaxes: O desacoplamento de servizos e clientes lévase a cabo a través de compoñentes intermedios que balancean a carga e fan funcións directorio, con risco de limitar o rendemento se non son suficientemente eficientes. O rendemento global é difícil de determinar, xa que pode depender moito de cada servizo.

Caracterización arquitectura cliente-servidor distribuído

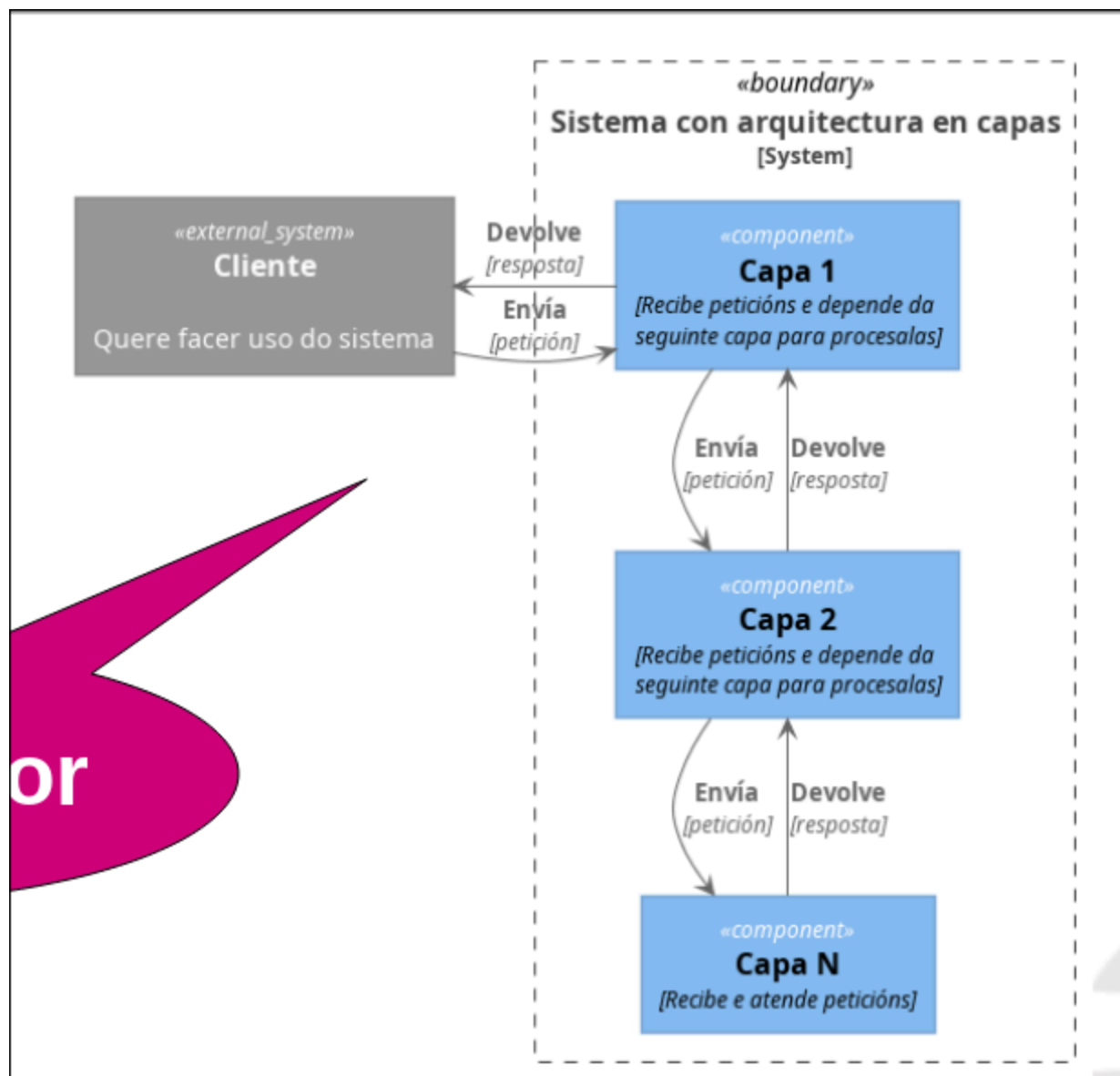
Descrición: As funcionalidades do sistema se organizan en servizos, cada un deles implementado por un compoñente dedicado, nun servidor específico. Os clientes do sistema usan estes servizos de xeito transparente.

Aplicabilidade: Úsanse cando se quere aumentar a robustez, escalabilidade ou rendemento do cliente/servidor clásico.

Vantaxes: Os servizos son independentes entre si, polo que poden duplicar para facer fronte a alta demanda, e o fallo dun (ou mesmo un fallo hardware) non debe afectar ao resto, de xeito que este estilo ofrece moi boa robustez. Tamén resulta doado ofrecer certos servizos a certos clientes, limitando a súa expansión.

Desvantaxes: O desacoplamento de servizos e clientes lévase a cabo a través de servidores intermedios que balancean a carga e fan funcións de directorio, con risco de limitar o rendemento se non son suficientemente eficientes. O rendemento global é difícil de determinar, xa que pode depender moito de cada servizo, e en grande medida tamén da canle de comunicacións.

Baséase na **organización dos compoñentes do sistema en etapas secuenciais (capas)**, que agrupan *responsabilidades* concretas, e de xeito que cada capa só recibe peticións da anterior e só envía peticións á seguinte. A maior **vantaxe** que presenta é a localización de cambios, xa que se unha capa cambia o cambio debe afectar só ás adxacentes. O maior **inconveniente** é que pode resultar **artificial** a organización de responsabilidades deste xeito; ademais, o feito de que as peticións percorran todas as capas **limita o rendemento**.



Caracterización da arquitectura en capas

Descrición	Organza o sistema en capas de responsabilidade concreta. Todas as capas participan na resolución de cada petición que se fai ao sistema. Cada capa só proporciona servizos á capa inmediatamente superior
Aplicabilidade	Úsase cando o fluxo de negocio ou lóxica de procesamento favorece esa división das reponsabilidades de xeito secuencial.
Vantaxes	Permítese substituír/reutilizar capas completas sempre que se manteña a interface coas capas adxacentes. A posibilidade de implementar mecanismos de seguridade específicos por capa pode incrementar a fiabilidade do sistema. É un estilo que adoita favorecer a portabilidade e o soporte multi-plataforma. Favorece tamén o reparto de traballo entre varios equipos de desenvolvemento ou entre persoas dun equipo.
Desvantaxes	Pode resultar difícil realizar unha "división horizontal" (por capas) das funcionalidades dun sistema. A necesidade de que cada petición atravesese todas as capas para ser procesada, e despois novamente para ser devolta, é unha limitación ao rendemento. Aínda así, non é recomendable que unha capa acceda a capas máis inferiores directamente, xa que isto conduce invariablemente ao deterioro da arquitectura.

Arquitectura pipe & filter

Baséase na organización dos compoñentes do sistema en **etapas secuenciais (filtros) de transmisión** de entradas en saídas

- Tamén se denomina arquitectura en *pipeline*
A maior **vantaxe** é que resulta doado engadir, eliminar, modificar ou mesmo duplicar etapas (filtros).
O maior **inconveniente** é que non resulta axeitado para sistemas interactivos, xa que a saída non se "devolve" pola mesma canle pola que se recibe a entrada/petición.

Caracterización da arquitectura pipe & filter

Descripción: A funcionalidade do sistema organízase nunha serie de pasos discretos de procesado (filtros) de información; as peticións/información pasan dun filtro a outro a través dunha canle ou compoñente/tubería (pipe), nunha única dirección. Todos os filtros participan na resolución de cada petición que se fai ao sistema.

Aplicabilidade: Sistemas adicados ao procesamento de información, onde é posible formalizar unha serie de etapas no procesado de cada petición para producir a saída desexada.

Vantaxes: O fluxo lóxico de negocio queda reflectido na propia arquitectura do sistema. Favorécese a reutilización de etapas (filtros), e a evolución do sistema (incorporación, modificación, substitución de filtros) e a división do traballo. Favorece o rendemento xa que non é preciso rematar de procesar por completo unha petición para comezar coa seguinte (só ten que rematar a primeira etapa); etapas pensadas poden duplicarse para executarse en paralelo.

Desvantaxes: As peticións entran e saen do sistema por localizacións diferentes, polo que non adoita ser axeitado para sistemas interactivos. A información que cada *pipe* leva dun filtro a outro debe estar nun formato válido para quen a produce e quen a recibe, evitando perder rendemento con conversións de saída/entrada.

Baséase na **facilitación do intercambio de información** entre compoñentes, que se canaliza a través dun **repositorio central compartido** que é accedido por todos.

A maior **vantaxe** é a minimización do número de envíos de información, especialmente relevante se a cantidade de información a compartir é grande.

O maior **inconveniente** é que o repositorio é un punto único de fallo.

Caracterización da arquitectura en repositorio

Descrición: Todos os datos do sistema se xestionan nun repositorio central accesible a todos os compoñentes. Os compoñentes do sistema non interactúan entre eles, só a través do repositorio.

Aplicabilidade: Sistemas nos que se precisa intercambiar gran cantidade de información, ou nos que a aparición ou modificación de información no repositorio funciona como disparador de accións nos diferentes compoñentes (*data-centered architecture, data-driven-systems*). Nos casos nos que os compoñentes do sistema se especializan en só producir ou só consumir información, úsase a nomenclatura produtores-consumidores.

Vantaxes: Os compoñentes do sistema son completamente independentes uns doutros, polo que os cambios neles nunca se afectan entre si. As políticas sobre os datos (por exemplo, *backups*) só se teñen que realizar sobre o repositorio compartido. Os cambios na información son inmediatamente visibles por todos os compoñentes.

Desvantaxes: O repositorio é un punto único de fallo, polo que a súa distribución ou sincronización cunha copia de seguridade pode ser problemática. O formato da información do repositorio debe ser válida para todos os compoñentes, para evitar perdas de rendemento en conversións.

Arquitectura líder-traballador

Baséase na presenza dun elemento (*líder*) **encargado da recepción e asignación de peticións** ao resto (*traballadores, centrados en tarefas específicas*), xunto coa súa xestión e coordinación.

A maior **vantaxe** é a separación e a serpaación clara entre as tarefas de xestión e as de execución de funcionalidades, que permite gran rendemento e escalabilidade.

O maior **inconveniente** é que o líder é un SPoF (*single point of failure*)

Caracterización da arquitectura líder-traballador

Descrición: As peticións chegan ao sistema a través dun compoñente líder que concentra as responsabilidades de xestión, tanto das interaccións cos clientes como do resto de compoñentes específicos (os traballadores).

Aplicabilidade: Este estilo resulta axeitado cando, ademáis da robustez que proporciona a diferenciación de tarefas administrativas vs. funcionalidades en máquinas diferentes, queremos ter a posibilidade de reacción elástica a cambios na carga/demanda do sistema.

Vantaxes: O arranque/parada de traballadores permite optimizar o uso de recursos dando unha axeitada resposta aos niveles de demanda do sistema en cada momento.

Desvantaxes: O líder é un punto único de fallo, polo que debe ser sometido a proba exhaustiva, técnicas de detección e recuperación ante erros, técnicas de reforzo da seguridade... para non comprometer a dispoñibilidade do sistema.

Baséase na **facilitación do reparto de traballo** entre **compoñentes**, que son todos **equivaletnes**, capaces de realizar as mesmas tarefas.

A maior **vantaxe** reside na súa gran capacidade para a dispoñibilidade, ao non ter puntos únicos de fallo, e a escalabilidade, ao poder incorporar compoñentes de xeito transparente.

O maior **inconveniente** reside na ausencia de control coordinado ou sincronización (non eventual) entre os compoñentes, e os posibles riscos de seguridade.

Caracterización da arquitectura peer-to-peer

Descrición: Sistemas descentralizados nos que calquera compoñente (par) pode atender una determinada petición recibida.

Aplicabilidade: Sistemas nos que os pares poden variar dinamicamente mais son quen de realizar as mesmas tarefas empregando a súa información local. Pode aparecer un compoñente específico (*super-peer*) para facilitar que os pares coñezan aos seus veciños e os clientes coñezan algún par.

Vantaxes: Aproveitamento óptimo dos recursos dispoñibles en cada momento no sistema (pares), configuración dinámica e auto-organizada do sistema.

Desvantaxes: Pode ser precisa unha aproximación semi-centralizada para simplificar os mecanismos ou protocolos de auto-descubrimentos entre pares, ou para introducir algunha medida de seguridade na entrada ao sistema de novos compoñentes/clientes. Os pares integran tanto lóxica de xestión (veciñanza) como lóxica de negocio (tarefas).

Reutilización

- Sistemas **COTS**
 - *Commercial Off-The-Shelf*
 - Compoñentes ou servizos de propósito xeral proporcionados por un terceiro
 - Aplicable a hardware e a software
 - Coste reducido
- Sistemas **legacy**
 - Sistemas herdados
 - Aplicable a hardware e a software
 - Amortización de custo

Reutilización por integración

Busca conseguir que aplicacións, servizos e/ou **compoñentes dispares traballen xuntos** para proporcionar un conxunto de **funcionalidades de maneira unificada**.

Características dun escenario de integración

- Combinación de elementos propios (*legacy*) e adquiridos a terceiros (*COTS*)
- Execución en diferentes máquinas físicas, plataformas diferentes, distintas ubicacións
- Non necesariamente deseñados para ser integrados, pouco flexibles.

Malia certas características xerais, as necesidades de integración varían en cada **escenario concreto** de integración

- Involucra unha serie de consideracións e consecuencias que deben terse en conta.
- É necesaria?
 - Se ñe posible desenvolver un sistema no que todos os compoñentes estean baixo o noso control, evitamos o problema da integración.
 - Porén, en moitas ocasións non é realista asumir o desenvolvemento de todos os compoñentes que se precisan (custo)
- Que **tipo de acoplamento** impón?
 - Debemos buscar minimizar as dependencias e asuncións para proporcionar a funcionalidade permitindo cambios nos compoñentes sen provocar a ruptura da integración.
- Que **esforzo** require?
 - A integración pode provocar cambios nos compoñentes ou, alternativamente, requirir o desenvolvemento de compoñentes específicos que a permitan

- Que **decisións tecnolóxicas** implica?
 - Se a integración require software/hardware especializado incrementará o seu custo, ou pode conducir a *vendor lock-in*
 - Desenvolver unha solución de integración propia en cada caso tamén pode ter un custo elevado e pode significar reinventar a roda continuamente.
- Como afecta ao **formato da información**?
 - Uno dun formato único
 - Cambiar compoñentes existentes para que usen un formato diferente pode ser inviable
 - Introducir un traductor intermediario facilita a evolución e/ou extensión de ese formato en caso de ser necesario
- Como afecta á **dispoñibilidade**?
 - Debemos minimizar o intervalo de tempo entre que un compoñente comparte certa información e outros poden acceder a ela
 - A segmentación da información pode acelerar a dispoñibilidade, pero pode introducir ineficiencias
- Integra **datos/funcionalidades/ambos**?
 - Algúns mecanismos de integración permiten un aspecto ou o outro e non ambos.
 - A integración de funcionalidades proporciona unha mellor abstracción de cara aos clientes, pero require mecanismos de integración máis complexos.
- Involucra **comunicación remota**?
 - Frente ao procesado síncrono tradicional, a integración de compoñentes remotos suxire a introducción de mecanismos asíncronos, necesarios para evitar bloqueos e mellorar a eficiencia, pero xeralmente máis complexos.
- Que **seguridade** proporciona?
 - A integración con compoñentes remotos é máis lenta e menos fiable, a dispoñibilidade de compoñentes locais é máis doada de garantir e a canle de comunicación local é máis segura
- **Non hai unha boa aproximación de integración para todos os caos**
- **Ningunha resolve todos os interrogantes anteriores de xeito equilibrado**

Estilos de integración

Transferencia de ficheiros

Os compoñentes xeran ficheiros coa información a compartir que son consumidos por outros compoñentes, e viceversa.

- Convención de nomeado e ubicación
- Política de unicidade de nomes

- Responsabilidade de borrado
 - Mecanismo de *locking* ou *sincronización*
 - Custe de procesado dos ficheiros
 - Protección de acceso
-
- Se precisa que os datos estean dispoñibles rapidamente e é posible acordar un formato único: *Compartición de fontes de datos*
 - Se se precisa integrar funcionalidade máis que datos: *Invocación de procedementos remotos*
 - Se o intercambio de datos é pequeno e ten como obxectivo solicitar a execución de servizos: *Paso de mensaxes*.

Compartición de fontes de datos

Os compoñentes usan un repositorio de datos onde almacenan a información que queren compartir

- Esquema de datos común
 - Punto crítico de rendemento
 - Distribución da fonte de datos (sincronización, rendemento)
 - Protección do acceso (usuarios, vistas)
-
- Se se precisa integrar funcionalidade máis que datos: *Invocación de procedementos remotos
 - Se o intercambio de datos é pequeno e interesa máis o tipo de dato en cada intercambio que utiliza un resquema global: *Paso de mensaxes*

Invocación de procedementos remotos

Cada compoñente pon a disposición do resto unha API de intercambio de datos/acceso a funcionalidades que pode invocarse de xeito remoto

- Cada compoñente pode ofrecer unha API diferente a distintos interlocutores
 - Risco de tratar as invocacións remotas como locais (latencia, dispoñibilidade)
 - Risco de *secuenciación* (imposición da orde nas chamadas)
 - Mecanismos de seguridade a nivel de compoñente
-
- Para integrar compoñentes de xeito desacoplado e asíncrono: *Paso de mensaxes*

Paso de mensaxes

Cada compoñente conéctase a unha canle de intercambio de mensaxes común, onde se intercambian datos/peticións de servizos

- A secuenciación evítase reordeando as mensaxes á hora de procesalas
- Probas e depuración son máis complexos
- Adaptadores máis complexos (máis que "tradutores de tecnoloxía", como na invocación de procedementos remotos)
- Definición de mensaxes, envío e recepción, tradución, canal, enrutados